



Joint optimization of order picking and replenishment in robotic mobile fulfillment systems

Jingwen Wu, Zhiyuan Yang^{*}, Lu Zhen, Wenxin Li, Yiran Ren

School of Management, Shanghai University, Shanghai, China

ARTICLE INFO

Keywords:

Warehouse management
RMFS
Order picking and replenishment
Variable neighborhood search

ABSTRACT

Advancements in intelligent warehousing have spotlighted the robotic mobile fulfillment system as a transformative solution for modern logistics challenges. This paper introduces a five-stage mixed-integer programming model designed to optimize the robotic mobile fulfillment system (RMFS) by minimizing the longest completion time, a critical metric in warehouse efficiency. Our comprehensive model strategically integrates the assignment of orders to stations and pods, the deployment of pods to robots, and the intricate details of route planning, order picking, and replenishment. Utilizing a variable neighborhood search algorithm, we not only tackle the complex scheduling decisions among orders, robots, stations, and pods but also demonstrate the model's effectiveness through rigorous numerical experiments. The results provide pivotal insights, revealing significant potential for enhancing the RMFS efficiency and offering practical guidance for warehouse managers.

1. Introduction

With the rapid development of e-commerce, the logistics industry is faced with challenges brought by the characteristics of wide customer demand, multi-variety orders, and small batches (Huang et al., 2023). In 2003, Kiva pioneered robotic mobile fulfillment systems (RMFS), which changed the traditional warehousing model (D'Andrea, 2012). The “parts-to-picker” model of RMFS greatly improves the speed of order processing. It is 2 to 3 times faster than traditional sorting systems and has improved picking efficiency, enhanced system scalability, and increased flexibility. (Lamballais et al., 2017). The essence of RMFS is its dynamic coordination of mobile robots and removable storage shelves (called pods). These robots efficiently shuttle through fulfillment centers, picking up items as needed and delivering them to stations. This innovative approach minimizes unnecessary manual handling, shortens operation time, and improves efficiency.

An RMFS consists of three main components: (1) a robotic infrastructure that includes robots, pods, and stations (picking and replenishment stations); (2) an autonomous control mechanism that enables functionalities such as collision avoidance, automated replenishment, efficient path planning, and speed modulation to mitigate congestion; and (3) sophisticated process control software that orchestrates the seamless interplay of orders, workstations, robotic units, and containers, leveraging IoT-driven data exchange networks for real-time information flow. Despite the relative maturity of the hardware and control technology (Xue and Li, 2023), the RMFS faces difficulties in managing diverse order structures (size and composition), complex allocation algorithms linking orders to

^{*} Corresponding author.

E-mail addresses: jingwen_wu@shu.edu.cn (J. Wu), zy217@shu.edu.cn (Z. Yang), lzhen@shu.edu.cn (L. Zhen), wqli0305@163.com (W. Li), reyran0216@163.com (Y. Ren).

<https://doi.org/10.1016/j.tre.2024.103930>

Received 5 March 2024; Received in revised form 12 November 2024; Accepted 6 December 2024

Available online 21 December 2024

1366-5545/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Pods, balancing item variety and distribution across pods, and strategically assigning robots and orders to their respective destinations, highlighting the operational complexities within this advanced logistics solution. These problems not only affect the efficiency of order processing but also increase the operating costs of enterprises. Therefore, studying how to optimize the RMFS picking system is of great significance for improving its efficiency. An essential challenge in the RMFS environment is efficiently assigning orders to picking stations while considering the varied material requirements of each order. Simultaneously, robots must be strategically dispatched to replenishment stations to collect the required items efficiently, and subsequently deliver them to the picking stations. The decision-making process involves complexities related to assigning orders to pods and picking stations rationally, planning efficient routes for robot transportation, and sequencing pods at the stations. To address these challenges, the development of a comprehensive mathematical model is imperative, considering the intricate interplay among order characteristics, spatial coordinates, robot capabilities, and station capacities.

Given the unique characteristics of the RMFS system, this paper focuses on jointly optimizing order allocation and path planning, while also considering the need for replenishment in order to address the aforementioned issues. Replenishment means that if a pod's stock-keeping units (SKUs) are insufficient to fulfil the order requirements, the pod needs to travel to the replenishment station to collect the required items. We propose a five-stage mixed-integer programming model to minimize the makespan, which represents the maximum completion time, considering the correlation and influence of operating time and queuing time. To efficiently solve this complex model, an enhanced variable neighborhood search (VNS) algorithm is developed, which provides a workable resolution to increase the RMFS's efficiency. The factors that affect the performance of the system are analyzed by sensitivity experiments, which provide theoretical guidance and practical enlightenment for the logistics center to realize the rational scheduling of storage resources.

The remaining structures of this paper are organized as follows: [Section 2](#) overviews a summary of the relevant literature. [Section 3](#) introduces the background of the problem in detail. [Section 4](#) and [Section 5](#) introduce a mathematical model and an algorithm. [Section 6](#) performs the numerical experiments and sensitivity analyses. [Section 7](#) summarizes and gives concluding remarks.

2. Related works

The optimization of RMFS has been rigorously examined across diverse domains, addressing multifaceted technical challenges from several viewpoints ([Masae et al., 2020](#); [Cergibozan et al., 2019](#)). This section delineates two principal streams of research within RMFSs: assignment decisions and integration optimization.

The first stream is about the assignment decisions in warehouses, which involve critical components such as order, robot, and pod allocation that directly influence warehouse operational efficiency and responsiveness. Substantial investigations have been undertaken on these issues. For instance, [Jiang et al. \(2020\)](#) introduce an MIP model aimed at synchronizing the processes of order picking and inventory replenishment within RMFSs. The authors posit that this integrated synchronization strategy could outperform traditional, disjointed methods under specific operational scenarios. Based on the varying processing speeds of workstations in RMFSs, [Zou et al. \(2017\)](#) introduce a new allocation rule and developed a local search strategy to approximate the optimal distribution scheme. To evaluate the system's performance, they employ semi-open queuing network models, combining them with a two-stage approximation methodology. [Xie et al. \(2021\)](#) incorporate the aspect of order splitting within an MIP model. Their focus was to optimize the allocation of orders and the selection of pods in RMFSs, addressing the complexities of these interconnected processes. To assess the operational efficiency, [Roy et al. \(2019\)](#) focus on the order retrieval and restocking activities within storage areas in an RMFS, utilizing multi-class closed queuing networks as the foundation. A two-stage stochastic model is proposed to determine optimal robot allocation across various storage zones. In addition, order assignment usually involves the order batching problem, which is an optimization problem belonging to the operational management aspect of a warehouse ([Menéndez et al., 2017b](#)). [Nicolas et al. \(2018\)](#) develop a model to optimize batch processing of orders by considering factors like order similarity and compatibility, which significantly reduces completion time. [Menéndez et al. \(2017b\)](#) introduce several strategies based on the VNS algorithm to tackle the order assignment and batching processes. By extending this study, [Menéndez et al. \(2017a\)](#) further propose a VNS-based heuristic approach to investigate the order batching and sequencing problem. The studies mentioned above focus on the optimization for the assignment of a single component. However, there are three main types of equipment used in the RMFS warehouse: stations, pods, and robots. The assignment of order-to-station, order-to-pod, and robot-to-pod are often intertwined. Their assignment should be decided simultaneously to increase the quality of the decision plan.

The second stream focuses on the integration optimization problems within RMFSs, particularly pertaining to the picking process, which encompasses order scheduling, equipment sequencing and path planning. More and more research is focusing on joint optimization problems within RFMS, which contributes to the overall efficiency of warehousing. For example, [Boysen et al. \(2017\)](#) construct an MIP model to address an operational-level decision issue pertaining to the synchronization of order picking and shelf rack moving in an RMFS context, with the inclusion of a single isolated pick station. Based on the MIP methodology, [Boysen et al. \(2023\)](#) further introduce an innovative methodology that unfolds in a two-step, multi-scenario design, tailored to enhance the real-time optimization of robotic sorting systems. [Yang et al. \(2021\)](#) present an MIP model and a two-stage heuristic addressing an integrated problem of order sequencing and pod scheduling in RMFSs. [Valle and Beasley \(2021\)](#) investigate an integrated problem of allocating orders and pods to pickers and the sequence of the pods. An MIP model and two heuristics are proposed for the problem-solving. [Wang et al. \(2022\)](#) investigate the challenge of devising an optimal robot scheduling strategy that is sensitively balanced against the variability in human picker's workloads caused by scheduling decisions. An algorithm based on approximate dynamic programming and branch-and-price is developed, tailored to facilitate efficient problem resolution. [Wang et al. \(2023\)](#) propose an integer programming model to jointly optimize the allocation of orders, order sequencing, the selection of pods, and pod sequencing. They also design a two-stage hybrid heuristic algorithm, where the first stage focuses on strategic order allocation, followed by a

second stage that concurrently addresses the sequencing of both orders and pods. Some studies also consider other factors that affect the operation of RFMS. For example, [Lamballais et al. \(2020\)](#) analyzed the impact of three key factors in RMFS: the number of pods per stock-keeping unit, the ratio of picking to replenishment stations, and the resupply level. [Dou et al. \(2015\)](#) and [Van Gils et al. \(2018\)](#) have integrated various aspects of RMFS to reflect real-world scenarios, examining the interplay between different strategies and their impact on picking efficiency.

Although the models and methods proposed in previous studies provide valuable insights and useful references for system optimization, it must be acknowledged that most of the existing studies mainly emphasize individual decisions and incomplete multi-objective decision-making. [Table 1](#) demonstrates that our study is significantly different from the above studies. This paper distinguishes itself by not only combining these disparate research streams into a single, coherent model but also by extending the analysis to consider the impacts of these integrated strategies on the operational efficiency of RMFS in a continuous-time setting. This holistic approach allows for a more nuanced understanding of system dynamics and operational interdependencies, paving the way for more robust and adaptable RMFS solutions.

3. Problem description

In the RMFS warehouse, a collaborative operation involving stations, pods, and robots optimizes the fulfillment of orders. Each order refers to the process of retrieving items from storage pods and is allocated to a picking station. The RMFS uses robots to lift movable pods and bring them to picking stations, and it is common for multiple stations to access the same pod consecutively. A pod stores different quantities and types of items. If a pod's SKUs are insufficient to fulfil the order requirements, the pod should be replenished at dedicated replenishment stations before the commencement of picking tasks. These replenishment stations are equipped to handle various item types, though the quantities they store are finite. Both replenishment and picking operations are structured as single-server queue systems, where only one pod can be serviced at a time. Consequently, when multiple pods queue at a station, it becomes imperative to strategize the sequence in which the pods are processed. The system's efficiency is measured by the completion time of the last order, which represents the maximum completion time of the system.

When a sufficient number of orders have accumulated in the order queue, they are allocated to the picking stations. Before the decision system can make this allocation, it must collect information about the locations of the pods, the SKUs stored in each pod and their quantities, and the current status and positions of the robots. In addition, the system requires configuration parameters relating to the travel time or distance of each link in the network, which consists of arcs connecting various points such as pod locations, station locations and robot positions. Using this data, the decision system solves mathematical models to efficiently decide the order-to-station, order-to-pod and robot-to-pod assignments, and plans the robot and pod paths. Once the instructions are issued, the decision system moves on to process the next batch of orders.

The mathematical models embedded in the decision system address an integrated optimization problem on the three main types of equipment used in the RMFS warehouse: stations (i.e., picking and replenishment stations), pods, and robots. The scheduling of these equipment is often intertwined. The design of the decision framework should take into account the interactions between these different types of equipment, and the decision variables could be mainly categorized into three aspects.

- (1) The assigning decision of orders to various equipment and the number of units of SKU to pods.
- (2) The sequencing decision for the three types of equipment to handle their assigned orders.
- (3) The timing decision for the three types of equipment to start the handling operations.

We propose a five-stage decision-making framework to systematically address the complexities of RMFS operations. The five-stage model is constructed to decompose the intricate integrated optimization problem into manageable sub-problems while capturing the critical interactions between stages. The primary objective of this integrated decision-making process is to minimize the makespan. The example in [Fig. 1](#) illustrates the overall decision framework and the interaction intertwined relationships among each stage.

Table 1
Comparison between this paper and studies.

Papers	Assignment decision			Sequencing decision			Model	Solution method
	Orders to station	Orders to pods	Robots to pods	Pods for each station	Stations for each pod	Pod replenishment		
Jiang et al. (2020)		✓				✓	MIP	VNS
Xie et al. (2021)	✓	✓					MIP	Simulation
Roy et al. (2019)			✓			✓	QM	Simulation
Boysen et al. (2017)		✓		✓			MIP	Heuristic
Yang et al. (2021)		✓		✓			MIP	Two-stage heuristic
Valle and Beasley (2021)	✓	✓		✓			MIP	Heuristic
Wang et al. (2022)		✓		✓			SDP	ADP and BP
Wang et al. (2023)	✓	✓		✓			IP	Two-stage heuristic
This paper	✓	✓	✓	✓	✓	✓	MILP	VNS

Notes: MIP: mixed integer programming; QM: queueing models; SDP: stochastic dynamic programming; IP: integer programming; MILP: mixed integer linear programming; ADP and BP: approximate dynamic programming and branch-and-price. VNS: variable neighborhood search.

In Stage 1, the objective is to create order clusters by grouping orders with similar SKU requirements and assigning them to picking stations. It is efficient to pick products from different orders together (Gzara et al. 2020). The clustering criteria is based on the similarity of the orders, and orders with identical SKUs are clustered. This approach reduces the variety of SKUs that each station needs to handle, which helps to streamline the picking process and balance the workload across stations. The way orders are paired with stations determines the SKU requirements at each station and has a direct impact on pod assignments in Stage 2. A poor assignment could lead to inefficient pod utilization and increased travel times.

In Stage 2, the SKUs stored in each pod are predetermined by the system. Pods are assigned to fulfill the SKU requirements of the orders, and replenishment is planned if necessary. Efficient pod assignment ensures that the required SKUs are available at the picking stations and minimizes the need for replenishment operations, which can be time-consuming. Decisions in this stage depend on the SKU demands from Stage 1. The order-pod pairs and replenishment plans influence the routing decisions in Stage 3, as pods need to visit replenishment stations before picking stations if replenishment is required.

In Stage 3, we focus on planning the routes for pods to reach their designated replenishment and picking stations. By optimizing these routes, we can reduce travel time and ensure that SKUs are delivered to picking stations on time, which is crucial for meeting order deadlines. The routes are determined based on the pod assignments and replenishment requirements identified in Stage 2. These planned routes also influence the robot assignments in Stage 4, as robots need to be scheduled to transport the pods along these specified paths.

In Stage 4, robots are assigned to pods, and their movement paths are planned to execute the pod routes. Efficient robot utilization ensures that the pods move on schedule, reducing idle and waiting times. Robot assignments depend on the pod routes from Stage 3. As the sequence of stations that a pod visits has been determined in Stage 3, the goal of Stage 4 aims to minimize the longest travel time between the original location of the robot and the storage location of the pod it is carrying.

In Stage 5, we concentrate on scheduling the processing of pods at both picking and replenishment stations. Because each station can accommodate only one pod at a time, effective scheduling is essential to prevent bottlenecks and minimize waiting times, which in turn directly affects the overall makespan. The arrival times of pods, which are determined by the routes and robot movements planned in Stages 3 and 4, serve as crucial inputs for this stage. Any delays in the earlier stages can cause congestion at the stations, necessitating adjustments in the schedule or sequence of pod processing.

Prior to presenting the mathematical model in Section 4, certain assumptions have been established as follows:

- (1) The items can be stored on any pod, and their storage locations in the pod are known.
- (2) Orders are not split, and each order can only be fulfilled by a picking station.
- (3) Picking operations begin only after all replenishment tasks have been completed.

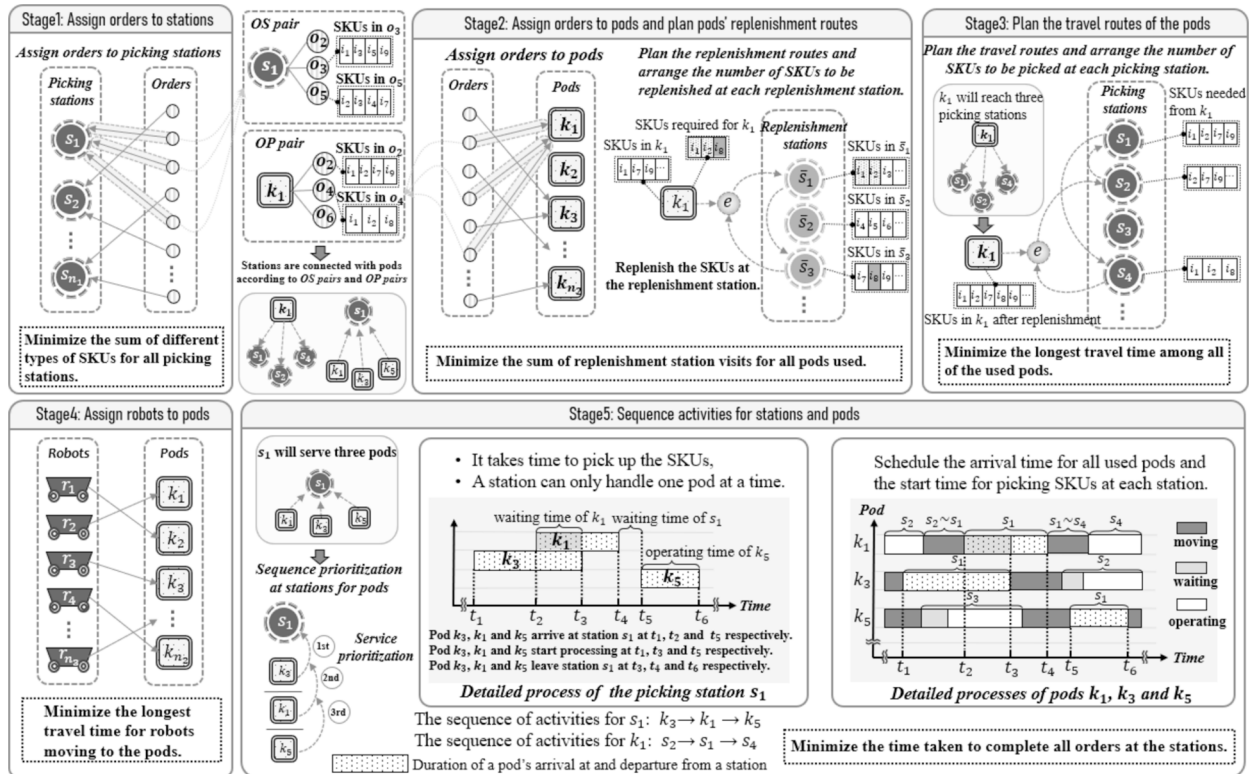


Fig. 1. General structure of the five-stage decision-making framework.

4. Mathematical model

To enhance the internal logistics efficiency of RMFS, a five-stage mixed integer linear programming (MILP) model is developed to address the aforementioned issues. In Section 4.1, a detailed presentation is provided for the model's parameters and decision variables. Section 4.2 further elaborates on the model adopted for addressing the problem. Next, Section 5 illustrates the solution algorithm for the model in the above method.

4.1. Notations

This section outlines the parameters and decision variables, with Roman letters representing parameters and Greek letters representing decision variables.

Indices and sets

O set of orders, with an index of o , $o \in \{1, \dots, |O|\}$.

I set of SKUs, with an index of i , $i \in \{1, \dots, |I|\}$

S set of the picking stations, with an index of s , $s \in \{1, \dots, |S|\}$.

$\bar{S}$ set of the replenishment stations, with an index of s , $s \in \{|S| + 1, \dots, |S| + |\bar{S}|\}$.

S_1 subset of $\bar{S}$, and $S_1 \subseteq \bar{S}$, $2 \leq |S_1| \leq |\bar{S}|$.

S_2 subset of S , and $S_2 \subseteq S$, $2 \leq |S_2| \leq |S|$.

K set of pods, with an index of by k , $k \in \{1, \dots, |K|\}$.

K_1 subset of K , $K_1 \subseteq K$ and $2 \leq |K_1| \leq |K|$.

R set of robots, with an index of r , $r \in \{1, \dots, |R|\}$.

e , e' , e'' dummy stations that are essential for standardizing the starting and ending points of travel routes for each pod within the RMFS. Specifically, e is used to denote the starting station for the picking phase of a pod's route and simultaneously acts as the ending station for the replenishment phase. e' is designated as the ending station of the picking phase. and e'' marks the starting station of the replenishment phase. These dummy stations are not physical locations but conceptual points being ideally positioned near the actual stations that pod visits.

$\acute{e}$, $\acute{e}'$ dummy pods that serve as the conceptual first and last pods in a series of pods visiting a station, respectively. These dummy pods are utilized to define the initiation and termination of pod visit sequences at each station.

Parameters

n_{oi} the number of units in SKU i , as demanded by order o .

m_{si} the number of the available units in SKU i in replenishment station s .

c_{ki} the number of units in SKU i in pod k .

c_{ki}^1 maximum number of units in SKU i that can be stored in a pod k .

c_{ki}^2 minimum number of units in SKU i that can be stored in a pod k . If the SKU stock falls below this specified threshold, it is automatically replenished to ensure that pod operations are not interrupted due to insufficient stock levels.

t_{ks}^1 travel time between the pod k and the station s .

$t_{ss'}^2$ travel time between the station s and s' .

t_{rk}^3 travel time between initial position of robot r and the designated pod k it services during an operational cycle.

a constant time unit for picking one SKU from the pod.

$F(i, x)$ function estimating the operating time to pick x units from SKU i . The function equals $\min\{x, 1\}$, if the SKU i is small-size; The function equals $\lceil x/2 \rceil$, if the SKU i is medium-size; The function equals $2x$, if the SKU i is large-size.

M a sufficiently large positive number.

Decision variables

α_{os} binary, if the order o is processed at the station s , α_{os} is equal to one, and otherwise zero.

β_{ok} binary, if the order o is satisfied by the pod k , β_{ok} is equal to one, and otherwise zero.

γ_{oki} number of the units of SKU i in pod k picked to fulfill the order o .

δ_{sk} binary, if the pod k visits the station s , δ_{sk} is equal to one, and otherwise zero.

ε_k binary, if the pod k is arranged, ε_k is equal to one, and otherwise zero.

ζ_{ski} number of the units of SKU i replenished in the pod k at the replenishment station s .

η_{ski} binary, if the units of SKU i of the pod k are replenished at the replenishment station s , η_{ski} is equal to one, and otherwise zero.

$\theta_{kss'}$ binary, if the pod k visits the station s and the station s' in sequence, $\theta_{kss'}$ is equal to one, and otherwise zero.

ϑ_{ks} the serial number of the pod k visits the station s .

σ_{rk} binary, if the robot r carries the pod k , σ_{rk} is equal to one, and otherwise zero.

μ_{ks} time when the pod k arrives at the station s .

ν_{ks} time when the pod k starts picking the SKUs at the station s .

ρ_{ks} the number of SKUs in the pod k picked at the station s .

ξ_{oki} binary, if SKU i from the pod k is selected for the order o , ξ_{oki} is equal to one, and otherwise zero.

$\varphi_{skk'}$ binary, if the pod k and k' visit station s in sequence, $\varphi_{skk'}$ is equal to one, and otherwise zero.

ϕ_{sk} the sequence index of the station s visited by pod k .

4.2. Model formulation

Utilizing the input parameters and decision variables outlined previously, we establish a five-stage MILP model for this study.

4.2.1. The model in stage 1 for assigning orders to picking stations

To optimize the RMFS system's efficiency and reduce the overall number of SKUs at picking stations, orders are initially assigned based on the degree of correlation between their SKUs in stage 1. The model determines how orders are grouped and assigns picking stations to complete each order.

$$\text{Minimize } \sum_{s \in S} \sum_{i \in I} \min \left\{ 1, \sum_{o \in O} n_{oi} \alpha_{os} \right\} \quad (1)$$

$$s.t. \sum_{s \in S} \alpha_{os} = 1 \quad \forall o \in O \quad (2)$$

$$\left\lfloor \frac{|O|}{|S|} \right\rfloor - 1 \leq \sum_{o \in O} \alpha_{os} \leq \left\lceil \frac{|O|}{|S|} \right\rceil \quad \forall s \in S \quad (3)$$

$$\alpha_{os} \in \{0, 1\} \quad \forall o \in O, s \in S \quad (4)$$

Objective (1) minimizes the sum of different types of SKUs for all picking stations. Constraints (2) guarantee that each order is allocated to only one picking station. Constraints (3) state that $|O|$ orders are grouped into $|S|$ picking stations (order clusters) and $\lceil |O|/|S| \rceil$ or $\lfloor |O|/|S| \rfloor - 1$ orders are included in each picking station (order cluster). Constraints (4) define the decision variables.

4.2.2. The model in stage 2 for assigning orders to pods and planning pod replenishment routing

During the second stage, all orders are allocated to the pod, and replenishment routes are planned. The picking operation starts after all replenishment tasks have been completed. Each order is strategically assigned to a suitable pod, considering the SKUs available within each pod. During the replenishment process, each pod makes informed decisions regarding its travel route, optimizing the selection and sequence of replenishment stations. It should be noted that Stage 2's decisions on pod allocation are influenced by the outcomes of Stage 1. Assigning orders to pods first allows for more controlled management of SKU types and quantities needed per station, thereby facilitating smoother replenishment operations.

$$\text{Minimize } \sum_{k \in K} \sum_{s \in \bar{S}} \delta_{sk} \quad (5)$$

$$s.t. \sum_{k \in K} \gamma_{oki} = n_{oi} \quad \forall o \in O, i \in I \quad (6)$$

$$\gamma_{oki} \leq M \beta_{ok} \quad \forall o \in O, k \in K, i \in I \quad (7)$$

$$\sum_{o \in O} \alpha_{os} \beta_{ok} \leq M \delta_{sk} \quad \forall s \in S, k \in K \quad (8)$$

$$\beta_{ok} \leq \varepsilon_k \quad \forall o \in O, k \in K \quad (9)$$

$$\delta_{sk} \leq \varepsilon_k \quad \forall s \in S, k \in K \quad (10)$$

$$\varepsilon_k \leq \sum_{s \in S} \delta_{sk} \quad \forall k \in K \quad (11)$$

$$\sum_{k \in K} \zeta_{ski} \leq m_{si} \quad \forall s \in \bar{S}, i \in I \quad (12)$$

$$\sum_{s \in \bar{S}} \zeta_{ski} + c_{ki} \leq c_{ki}^1 + M(c_{ki} - c_{ki}^2)^+ \quad \forall k \in K, i \in I \quad (13)$$

$$\sum_{s \in \bar{S}} \zeta_{ski} + c_{ki} \geq c_{ki}^1 - M(c_{ki} - c_{ki}^2)^+ \quad \forall k \in K, i \in I \quad (14)$$

$$\sum_{o \in O} \gamma_{oki} \leq \sum_{s \in \bar{S}} \zeta_{ski} + c_{ki} \quad \forall k \in K, i \in I \quad (15)$$

$$\zeta_{ski} \leq M \eta_{ski} \quad \forall k \in K, s \in \bar{S}, i \in I \quad (16)$$

$$\eta_{ski} \leq \zeta_{ski} \quad \forall k \in K, s \in \bar{S}, i \in I \quad (17)$$

$$\eta_{ski} \leq \sum_{s' \in \bar{S} \cup \{e''\}} \theta_{ks's} \quad \forall k \in K, s \in \bar{S}, i \in I \quad (18)$$

$$\sum_{s' \in \bar{S} \cup \{e\}} \theta_{kss'} = \sum_{s' \in \bar{S} \cup \{e''\}} \theta_{ks's} = \delta_{sk} \quad \forall k \in K, s \in \bar{S} \quad (19)$$

$$\sum_{s \in \bar{S} \cup \{e''\}} \theta_{kse} = \varepsilon_k \quad \forall k \in K \quad (20)$$

$$\sum_{s \in \bar{S} \cup \{e\}} \theta_{ke''s} = \varepsilon_k \quad \forall k \in K \quad (21)$$

$$\sum_{s \in \bar{S}_2} \sum_{s' \in \bar{S}_2} \theta_{kss'} \leq |S_1| - 1 \quad \forall k \in K, S_1 \subseteq \bar{S}, 2 \leq |S_1| \leq |\bar{S}| \quad (22)$$

$$\beta_{ok} \in \{0, 1\} \quad \forall o \in O, k \in K \quad (23)$$

$$\gamma_{oki} \geq 0 \quad \forall o \in O, k \in K, i \in I \quad (24)$$

$$\delta_{sk} \in \{0, 1\} \quad \forall k \in K, s \in \bar{S} \quad (25)$$

$$\varepsilon_k \in \{0, 1\} \quad \forall k \in K \quad (26)$$

$$\zeta_{ski} \geq 0 \quad \forall k \in K, s \in \bar{S}, i \in I \quad (27)$$

$$\eta_{ski} \in \{0, 1\} \quad \forall k \in K, s \in \bar{S}, i \in I \quad (28)$$

$$\theta_{kss'} \in \{0, 1\} \quad \forall k \in K, s \in \bar{S} \cup \{e''\}, s' \in \bar{S} \cup \{e\} \quad (29)$$

Objective (5) minimizes the sum of replenishment station visits for all pods used. Constraints (6) determine that all the SKUs demanded in orders can be satisfied by the pod arranged for those orders. Constraints (7) combine the two decision variables associated with assigning the order o to the pod k . If the number of units of SKU i of the pod k picked to contented the order o is greater than 0, then order o selects pod k . In other words, when γ_{oki} is greater than 0, β_{ok} is equal to 1. The big-M in Constraints (7) can be set as the maximum number of units in SKU that can be stored in a pod. Constraints (8) connect the two assignment decisions, i.e., the allocation of orders to stations and orders to pods. $\sum_{o \in O} \alpha_{os} \beta_{ok}$ is the total number of orders completed by the picking station s and served by pod k . If the value of $\sum_{o \in O} \alpha_{os} \beta_{ok}$ equal to 0 then δ_{sk} is 0, and if the value is greater than 0 then δ_{sk} is 1. The big-M in Constraints (8) can be set as the number of orders $|O|$. Constraints (9) export the decision variable ε_k , which signifies the arrangement status of a pod. Constraints (10) determine that when the pod k is not arranged, δ_{sk} equals 0. Constraints (11) determine that when the pod k is not arranged, ε_k equals 0. Constraints (12) determine that the number of replenished units does not surpass the available number in the replenishment station. Constraints (13) and (14) state that when the number of SKU i in the pod k is below the threshold c_{ki}^2 , the units for SKU i should be replenished to its maximum number in the pod. Constraints (13) state that the number of units in a pod (after the replenishment) should be lower than the pod's capacity. Constraints (14) ensure that that replenishment tasks are completed effectively, maintaining stock levels above the threshold c_{ki}^2 . When the stock level of SKU i in pod k falls below the threshold, the model triggers a replenishment action to restore the stock level to c_{ki}^1 . Constraints (15) ensure that the number of units selected in the pod will not surpass the available number in the pod (after the replenishment). Constraints (16)–(18) ensure that a pod can be replenished when it visits a replenishment station. The big-M in Constraints (16) can be set as the maximum number of units in SKU that can be stored in a pod. Constraints (19)–(21) delimit the pods' routes. Constraints (22) remove sub-tours. Constraints (23)–(29) define the decision variables.

4.2.3. The model in stage 3 for planning the travel route of the pods

In order to minimize the maximum travel time among pods, the next step is to map out the travel route to the picking stations for each pod. Based on the outputs of the previous two stages the relationship between pods and picking stations can be determined to get the set of picking stations visited by each pod.

$$\text{Minimize } \max_{k \in K} \left\{ \sum_{s \in S} t_{ks}^1 \theta_{ke''s} + \sum_{s \in S} \sum_{s' \in S} t_{ss'}^2 \theta_{kss'} + \sum_{s \in S} t_{ss}^2 \theta_{kse'} \right\} \quad (30)$$

$$\text{s.t. } \sum_{s' \in S \cup \{e\}} \theta_{kss'} = \sum_{s' \in S \cup \{e\}} \theta_{ks's} = \delta_{sk} \quad \forall k \in K, s \in S \quad (31)$$

$$\sum_{s \in S} \theta_{kse'} = \varepsilon_k \quad \forall k \in K \quad (32)$$

$$\sum_{s \in S} \theta_{kes} = \varepsilon_k \quad \forall k \in K \quad (33)$$

$$\sum_{s \in S_2} \sum_{s' \in S_2} \theta_{ks's} \leq |S_2| - 1 \quad \forall k \in K, S_2 \subseteq S, 2 \leq |S_2| \leq |S| \quad (34)$$

$$\vartheta_{ks'} \geq \vartheta_{ks} + 1 - M(1 - \theta_{ks's}) \quad \forall k \in K, s \in S \cup \{e\}, s' \in S \cup \{e'\} \quad (35)$$

$$\theta_{ks's} \in \{0, 1\} \quad \forall k \in K, s \in S \cup \{e\}, s' \in S \cup \{e'\} \quad (36)$$

$$\vartheta_{ks} \in \mathbb{Z}^+ \quad \forall k \in K, s \in S \cup \{e, e'\} \quad (37)$$

Objective (30) is to minimize the maximum travel time for each pod. Constraints (31)–(33) build the travel route for all the arranged pods. Constraints (34) remove possible sub-tours. Constraints (35) construct the sequence of pod visits to picking stations. The big-M in Constraints (35) can be set as $|S| + 1$. Constraints (36) and (37) define the decision variables.

4.2.4. The model in stage 4 for assigning robots to pods

According to the order allocation of pods, each pod to be moved is assigned a robot. As the sequence of pod visiting stations is already set, the goal of Stage 4 is to minimize the longest travel time between the original location of the robot and the storage location of the pod it is carrying.

$$\text{Minimize } \max_{r \in R} \left\{ \sum_{k \in K} t_{rk}^3 \sigma_{rk} \right\} \quad (38)$$

$$\text{s.t. } \sum_{k \in K} \sigma_{rk} \leq 1 \quad \forall r \in R \quad (39)$$

$$\sum_{r \in R} \sigma_{rk} = \varepsilon_k \quad \forall k \in K \quad (40)$$

$$\sigma_{rk} \in \{0, 1\} \quad \forall r \in R, k \in K \quad (41)$$

Objective (38) minimizes the longest travel time between a robot's original location and the storage location of the target pod. Constraints (39) and (40) ensure a one-to-one mapping between pods and robots, with each used pod being exclusively carried by a single robot. Constraints (41) define the decision variables.

4.2.5. The model in stage 5 for sequencing prioritization at stations for pods

In the final stage, the activities of each station and each pod are sequenced. In order to minimize the maximum completion time, we determine the arrival time of all used pod holds and the pick-up start time at each station. The value obtained in this stage is taken as the final objective value.

$$\text{Minimize } \max_{s \in S} \{\nu_{e's}\} \quad (42)$$

$$\text{s.t. } \sum_{k \in K \cup \{e\}} \varphi_{ske'} = 1 \quad \forall s \in S \cup \bar{S} \quad (43)$$

$$\sum_{k \in K \cup \{e'\}} \varphi_{se'k} = 1 \quad \forall s \in S \cup \bar{S} \quad (44)$$

$$\sum_{k' \in K \cup \{e'\}} \varphi_{skk'} = \sum_{k' \in K \cup \{e\}} \varphi_{sk'k} = \delta_{sk} \quad \forall k \in K, s \in S \cup \bar{S} \quad (45)$$

$$\sum_{k' \in K_1} \sum_{k \in K_1} \varphi_{skk'} \leq |K_1| - 1 \quad \forall s \in S \cup \bar{S}, K_1 \subseteq K, 2 \leq |K_1| \leq |K| \quad (46)$$

$$\phi_{sk'} \geq \phi_{sk} + 1 - M(1 - \varphi_{skk'}) \quad \forall k \in K \cup \{e\}, k' \in K \cup \{e'\}, s \in S \quad (47)$$

$$\varphi_{skk'} \leq \xi_{oki} \quad \forall o \in O, k \in K, i \in I \quad (48)$$

$$\rho_{ks} = \sum_{i \in I} \min \left\{ \sum_{o \in O} \alpha_{os} \xi_{oki}, 1 \right\} \quad \forall k \in K, s \in S \quad (49)$$

$$\rho_{ks} = \sum_{i \in I} \eta_{ski} \quad \forall k \in K, s \in \bar{S} \quad (50)$$

$$\mu_{ks} \leq \nu_{ks} \quad \forall k \in K, s \in S \cup \bar{S} \cup \{e\} \quad (51)$$

$$\sum_{r \in R} t_{rk}^3 \sigma_{rk} + t_{ks}^1 \leq \mu_{ks} + M(1 - \theta_{ke's}) \quad \forall k \in K, s \in \bar{S} \quad (52)$$

$$\sum_{r \in R} t_{rk}^3 \sigma_{rk} + t_{ks}^1 \leq \mu_{ks} + M(2 - \theta_{ke^r e} - \theta_{kes}) \quad \forall k \in K, s \in S \quad (53)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F(i, \zeta_{ski}) + t_{ks}^1 + t_{ks'}^1 \leq \mu_{ks'} + M(2 - \theta_{kse} - \theta_{kes'})$$

$$\forall k \in K, s \in \bar{S}, s' \in S \cup \{e'\} \quad (54)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F(i, \zeta_{ski}) + t_{ss'}^2 \leq \mu_{ks'} + M(1 - \theta_{kss'})$$

$$\forall k \in K, s \in \bar{S}, s' \in \bar{S} \quad (55)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F\left(i, \sum_{o \in O} \alpha_{os} \gamma_{oki}\right) + t_{ss'}^2 \leq \mu_{ks'} + M(1 - \theta_{kss'})$$

$$\forall k \in K, s \in S \cup \{e\}, s' \in S \cup \{e'\} \quad (56)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F(i, \zeta_{ski}) \leq \nu_{k's} + M(1 - \varphi_{skk'})$$

$$\forall k \in K \cup \{\dot{e}\}, k' \in K \cup \{\dot{e}'\}, s \in \bar{S} \quad (57)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F\left(i, \sum_{o \in O} \alpha_{os} \gamma_{oki}\right) \leq \nu_{k's} + M(1 - \varphi_{skk'})$$

$$\forall k \in K \cup \{\dot{e}\}, k' \in K \cup \{\dot{e}'\}, s \in S \quad (58)$$

$$\mu_{ks}, \nu_{ks}, \rho_{ks} \geq 0 \quad \forall k \in K \cup \{\dot{e}, \dot{e}'\}, s \in S \cup \bar{S} \cup \{e, e'\} \quad (59)$$

$$\xi_{oki} \in \{0, 1\} \quad \forall o \in O, k \in K, i \in I \quad (60)$$

$$\varphi_{skk'} \in \{0, 1\} \quad \forall s \in S, k \in K \cup \{\dot{e}\}, k' \in K \cup \{\dot{e}'\} \quad (61)$$

$$\phi_{sk} \in \mathbb{Z}^+ \quad \forall s \in S, k \in K \cup \{\dot{e}, \dot{e}'\} \quad (62)$$

Objective (42) minimizes the makespan for completing all orders at the stations. Constraints (43)–(45) establish the sequence of pod visits to each station. Constraints (46) remove possible sub-tours. Constraints (47) construct the service sequence of the picking station to pods. The big-M in Constraints (47) can be set as $|K| + 1$. Constraints (48)–(50) are designed to count the quantity of SKUs selected from pod k at station s , in order to determine the operating time. Constraints (51)–(58) delimit the operation time and sequence at specific time points. Constraints (51) guarantees that the arrival time is before the operation's start time when the pod k is at station s . Constraints (52) and (53) restrict pods from matching robots before going to the station. Constraints (54)–(56) associate the start time and arrival time of a pod passing through two consecutive stations. The time interval is composed of the traveling time from stations s to s' (i.e., $t_{ss'}^2$) and the operating time (i.e., $a\rho_{ks} + \sum_{i \in I} F(i, \sum_{o \in O} \alpha_{os} \gamma_{oki})$). Constraints (57) and (58) establish a connection between the beginning operating time of the current pod and the subsequent pod at the station. The time interval corresponds to the operating time (i.e., $a\rho_{ks} + \sum_{i \in I} F(i, \zeta_{ski})$ and $a\rho_{ks} + \sum_{i \in I} F(i, \sum_{o \in O} \alpha_{os} \gamma_{oki})$ represent the operating time at the picking station and the replenishment station, respectively). Constraints (59)–(62) define the decision variables.

Constraints (8), (49), (56), and (58) are nonlinear as they contain the product of two variables. They are linearized by adding some variables and constraints. Appendix A describes the details.

5. Solution methodology

The proposed five-stage MILP model can be solved using commercial solvers. However, due to the presence of multiple NP-hard problems, these solvers struggle to efficiently handle large-scale problems within an acceptable time. Hence, a highly efficient solution for solving complex large-scale multi-stage problems is provided by the VNS algorithm, which has been extensively validated.

5.1. General framework

The algorithm for the RMFS problem optimally integrates order allocation, replenishment scheduling, path planning, and service sequencing. Since the problem is difficult to solve directly, it is divided into five phases. First, assign orders to picking stations. Second, bind orders to pods and develop replenishment schedules. Third, schedule routes for each pod. Then, match the robot with the pod. Finally, arrange the service order of the pods if their travel routes overlap.

The paper uses a combinatorial approach to obtain a final generic solution for solving large-scale problems. Specifically, with the

order assignment scheme confirmed, we can acquire the stations each pod will pass through and the number of SKU units required. We can obtain the route of each pod and match it with the best robot. We will then consider the order in which pods arrive at each station under the time window and determine the value of the solution using the objective function.

From the above, when given the combination of order assignment to stations and pods during RMFS execution, we can get a solution (replenishment plan, robot selection, pod route, and start service time under this assignment) and its objective value. The algorithm is based on the VNS framework, which is a local search method designed to tackle and solve global optimization problems. The VNS algorithm employs a diverse set of actions within the neighborhood structure to conduct alternating searches, aiming to achieve the global optimal solution.

Algorithm 1. (Illustrates the details of the procedure. There are two stopping conditions: (1) The number of iterations is reached. (2) There is no better solution within the specified number of iterations.)

Algorithm 1: A VNS algorithm based on the five-stage model for RMFS

Input: set of neighborhood structures N_l for $l = 0, \dots, l_{max}$ for local search,
 set of neighborhood structures N_k for $k = 0, \dots, k_{max}$ for local search,
 maximum number of iterations $iter_{max}$, number of un-updated m .

```

1: Generate the initial solution  $\pi$ 
2: While  $iter \leq iter_{max}, num \leq num_{max}$  do
3:   For  $k = 0 \leftarrow k_{max}$  do
4:     Pick a random solution  $\pi'$  from the  $k^{th}$  neighborhood  $N_k(\pi)$  of  $\pi$ 
5:     For  $l = 0 \leftarrow l_{max}$  do
6:       Find the best neighbor  $\pi''$  of  $\pi'$  in  $N_l(\pi')$ 
7:       If  $f(\pi'') < f(\pi')$  then
8:          $\pi' = \pi'', l = 0$ ;
9:       Else
10:         $l = l + 1$ ;
11:       End if
12:     End for
13:     If  $f(\pi') < f(\pi)$  then
14:        $\pi' = \pi'', k = 0, num = 0$ ;
15:     Else
16:        $k = k + 1, num = num + 1$ ;
17:     End if
18:   End for
19: End while
20: Return optimal solution  $\pi$ 

```

During the implementation of the algorithm, some key questions arise: How to obtain high-quality initial solutions? How to rationally arrange orders with pods and stations? How to quickly find the best route and service order of pods to avoid falling into local optimality? We discuss these three issues in detail in subsections 5.2–5.4.

5.2. A simple rule-based algorithm to obtain the initial solution

Based on the properties of the RMFS problem, a simple heuristic rule is created to produce the initial overall solution.

Dividing the order as evenly as possible between each station to keep the total operating time at each station the same as possible. Based on the order information, we can calculate the operation time ($\sum_{i \in I} F(i, \sum_{o \in O} n_{oi})$) for each order. Orders are assigned in descending time-consuming sequence. We assign the current order to the station with the lowest current total elapsed time (assigned at random when there is more than one minimum). Replenishment is undesired; it not only increases the pod traveling path but also increases the operation time to pick up SKUs. If the current SKU i of pod k does not satisfy the required SKU i of the order o (i.e., $n_{oi} \geq c_{ki}$), we calculate the replenishment time based on the function ($\sum_{i \in I} F(i, \sum_{o \in O} (n_{oi} - c_{ki}))$) and assign the order o to the pod k that takes the shortest replenishment time. Based on the order allocation results, we can establish a connection between pods and stations with C_p (the collection of stations the pod k is going to). We use greedy criteria to organize the pod routing planning. First, calculate the distance between pod k and all stations in C_p . Afterwards, choose the station with the shortest distance as the next point and remove this station from C_p . Repeating the procedure until C_p is empty, we get a complete path for pod k . We use the first-come-first-served principle to sort the sequence of pods in the station. One station can only serve one pod at a time. If the station is occupied, subsequently arriving pod k' must wait for the current pod k to leave before starting service. The start time of pod k is equivalent to the completing time of pod k' (i.e., $\nu_{k's} = \nu_{ks} + a\rho_{ks} + \sum_{i \in I} F(i, \sum_{o \in O} \alpha_{os} \gamma_{oki})$). Combining these results, we get the initial solution for the next step of the algorithm.

5.3. Several neighborhoods to get different solutions

Four neighborhoods $N_l(\pi)$ are specifically designed to obtain distinct sets of solutions through the implementation of varied strategies. The first two strategies ($l = 0, 1$) are used to get different orders-stations and orders-pods correspondences by adjusting the

values of α_{os} and β_{ok} . The latter two strategies ($l = 2, 3$) are used to get different pod routes by adjusting the value of $\theta_{kss'}$.

(1) Sequencing exchange strategy. If order o was originally assigned to station s and pod k , this strategy randomly reallocates order o to the next station $s + 1$ or the next pod $k + 1$. For example, before implementing the sequencing exchange strategy, order o_3 was assigned at station s_2 . By applying this strategy to order o_3 , it is now completed at station s_3 , resulting in the value of $\alpha_{o_3s_2}$ changing from 1 to 0, and the value of $\alpha_{o_3s_3}$ changing from 0 to 1.

(2) Pairwise exchange strategy. For an order assignment scheme, this strategy involves swapping the station s or pod k selected by order o with the station s' or pod k' selected by order $o + 1$. For example, before implementing the pairwise exchange strategy, order o_2 was completed at station s_1 and order o_3 at station s_2 . After applying this strategy to orders o_2 and o_3 , order o_2 is now completed at station s_2 and order o_3 at station s_1 . Consequently, the values of $\alpha_{o_2s_1}$ and $\alpha_{o_3s_2}$ change from 1 to 0, and the values of $\alpha_{o_2s_2}$ and $\alpha_{o_3s_1}$ change from 0 to 1.

(3) Mirror inversion strategy. This strategy achieves the mirror reversal of part of the pod's routes. Specifically, when two stations are selected in a pod's route, the order of stations between them is reversed, and the unselected parts remain unchanged. For example, the route of pod k_1 is: $s_5, s_3, s_1, s_2, s_8, s_4, s_6$. Selecting the route between station s_3 and s_8 for the mirror inversion strategy, and the route of pod k_1 is adjusted to $s_5, s_8, s_2, s_1, s_3, s_4, s_6$, where the values of $\theta_{k_1s_3s_1}$, $\theta_{k_1s_1s_2}$, and $\theta_{k_1s_2s_8}$ change from 1 to 0, and $\theta_{k_1s_1s_3}$, $\theta_{k_1s_2s_1}$, and $\theta_{k_1s_8s_2}$ change from 0 to 1.

(4) Parallel movement strategy. This strategy achieves parallel movement of part of the pod's routes. Specifically, when two stations are selected in a pod's route, the stations between them are moved to the front of the route, while the remaining unselected parts are moved sequentially afterwards. For example, if the route of pod k_1 is: $s_5, s_3, s_1, s_2, s_8, s_4, s_6$, selecting the stations between station s_3 and s_8 for the parallel movement strategy, the route of pod k_1 is adjusted to: $s_3, s_1, s_2, s_8, s_5, s_4, s_6$, where the values of $\theta_{k_1s_5s_3}$ and $\theta_{k_1s_5s_4}$ change from 1 to 0, and $\theta_{k_1s_8s_5}$ and $\theta_{k_1s_5s_6}$ change from 0 to 1.

Based on the neighborhood structures outlined previously, we developed a VND framework to search for an improved solution, with the pseudo-code detailed in Appendix B.

5.4. Several stochastic strategies for escaping local optimum

Before starting the VND process, the algorithm will carry out the shaking operation to perturb the current local best solution. Shaking is designed to jump out of the local best solutions currently found and introduce more diversity in a random and exploratory way to further explore the solution space. We designed two shaking movements with different strategies for different stages.

(1) Bold randomized strategy. In the early stage of the algorithm, we adopt a bold random strategy to perform a wide range of searches. We believe that the overall change can accelerate the convergence of the algorithm and explore more unknown solution space. In this strategy, we have a p^1 probability of exchanging the assigned station for each order. Assigning order o to the new picking station s' (let $\alpha_{os'} = 1$) randomly and cancelling the original picking station s (let $\alpha_{os} = 0$). Similarly, we set the p^2 probability to change the pod allocation per order, cancel the assigned pod k (let $\beta_{ok} = 0$) and randomly assign order o to the new pod k' (let $\beta_{ok'} = 1$). We set the probability $p^1 = p^2 = \frac{\bar{n}}{2} \times \text{iteration}$. $\bar{n}$ indicates the number of times that have not been updated continuously. iteration represents the total number of iterations.

(2) Cautious randomized strategy. In the late stage of the algorithm, we adopt a cautious random strategy to carry out a refined search. We assume that the current solution is close to the optimum and that only small changes are needed to adjust the structure of the solution. In this strategy, we have p^3 probability of randomly choosing order o and order o' to swap their stations. Similarly, we have a p^4 probability of swapping on pods. We set the probability $p^3 = p^4 = \frac{\widetilde{\text{iter}}}{2} \times \text{iteration}$, $\widetilde{\text{iter}}$ denotes the current number of iterations.

6. Numerical experiments

This section presents a comprehensive set of randomly generated numerical experiments to assess the effectiveness of the VNS algorithm. Additionally, sensitivity experiments are carried out to provide managerial insights. The experiments were performed on a workstation equipped with an Intel i7-7700HQ CPU @ 2.80 GHz and 12 GB RAM. The operating system used was Windows 10. The model and algorithms were programmed in C# (VS 2019) employing IBM ILOG CPLEX 12.6.1. The time limit of 7,200 s is set for all instances.

6.1. Test parameters generation

A set of instances was created to replicate various scenarios in RMFS. The RMFS is simulated using a set of experimental parameters to replicate various scenarios. These parameters mainly include the coordinates of the picking stations, the coordinates and inventory of the replenishment stations, the operating speed of the workers, the coordinates and speed of the robots, the coordinates and capacity of the pods, and the content of the orders.

The stations, robots, and pods are distributed across a 100 square kilometer area. Stations are uniformly positioned in the range of $[0, 80]$, while robots are allocated within the range of $[81, 100]$, and pods are randomly placed throughout the entire area of $[0, 100]$. Distances between any two points are calculated using the Manhattan distance metric. The robots' transportation speed is set at 1 m per second (m/s). Travel time parameters t_{ks}^1 , $t_{ss'}^2$, and t_{rk}^3 are derived based on the spatial and speed parameters specified above. SKU demand distribution is modelled on current order data, with the proportions of small, medium, and large-sized SKUs set at 50 %, 30 %, and 20 %.

and 20 % respectively. The quantity of SKUs stored in each pod is determined using a discrete uniform distribution, $U[0,10]$. Similarly, the number of SKUs required per order is generated by $U[0,10]$, and the supply parameter m_{si} by $U[0,50]$. Each SKU i in pod k , can store a maximum of 20 units ($c_{ki}^1 = 20$), following the uniform distribution $U[0,20]$. SKUs that fall below a threshold of 5 units are replenished ($c_{ki}^2 = 5$). The workers' operational speed a is established at 5 s per task, reflecting the typical rate at which SKUs are handled.

We utilized the parameter settings provided above to create nine instance groups (ISGs) to assess the VNS algorithm's effectiveness. Table 2 shows the essential parameter settings for various ISGs, categorized as small, medium, and large-scale instances, denoted as ISG1-ISG3, ISG4-ISG6, and ISG7-ISG9, respectively. These parameter settings can simulate a variety of different scenarios in the RMFS environment. The performance and effectiveness of the VNS algorithm will be evaluated in subsequent experiments and analyses using these parameter values.

6.2. Computational results of the algorithm

As shown in Table 3, we thoroughly compare the VNS method with the CPLEX solver in small-scale cases. Additionally, the comparison results between VNS and Rule in medium-scale and large-scale instances are demonstrated in Table 4 and Table 5, respectively.

6.2.1. Small-scale experimental results

To assess the effectiveness of the CPLEX and the VNS algorithm, a comparative analysis was performed using randomly generated instances from the ISG1, ISG2, and ISG3 groups. When dealing with small-scale problems, the number of orders, stations, and pods considered is relatively small and both CPLEX and VNS algorithms succeed in obtaining the best solutions and show satisfactory solution speeds in ISG1, as shown in Table 3. However, upon analyzing the outcomes from ISG2 and ISG3, it becomes evident that while CPLEX still attains the best solution, its solution speed considerably decreases. Furthermore, CPLEX's solution time is noticeably slower than VNS's as the problem size increases. Notably, for larger problem sizes, the solution time of CPLEX exceeds 7,200 s. Conversely, the proposed VNS algorithm exhibits the capability to identify the global best solution within a shorter timeframe and with a smaller optimality gap. Consequently, the VNS algorithm proves to be more suitable for solving small-scale instances.

6.2.2. Medium-scale experimental results

To further validate the performance of the VNS algorithm for solving medium-scale problems, three instance groups were used, and the solutions obtained from the algorithm were compared with those derived from the introduced decision rules. Table 4 presents the results of the medium-scale experiment. The VNS algorithm demonstrates its superiority in terms of time, as it takes no more than 400 s to reach a solution. In terms of solution quality, the VNS algorithm is superior to Rule, with an average optimality gap of 23.10 %. The comparison highlights its advantages over the introduced decision rules, further validating its suitability for practical applications.

6.2.3. Large-scale experimental results

To prove the efficiency of the VNS algorithm in dealing with large-scale optimization problems in more depth, we further enlarge the relevant parameters of the instance groups. Table 5 provides a comparative analysis of the results obtained from these experiments, which show that in large-scale instances, the VNS algorithm produces higher-quality solutions than existing decision rules. Notably, the average optimality gap value for GAP was found to be 27.79 %, which is larger than that observed in the medium-scale (23.10 %) and small-scale (1.07 %) instances, thus indicating the stability and efficacy of the algorithm in obtaining the best possible solution across varying instance sizes. The presented comparative analysis serves to highlight the superiority of the VNS algorithm, which is capable of discovering the global best solution within a specified time frame and with a computation time averaging less than 1,000 s.

6.3. Sensitivity analysis

Sensitivity analysis is crucial for identifying the key factors that impact system performance and for providing managerial insights to optimize the operational workflows of RMFS. In this subsection, we examine the effects of two critical factors: order composition and

Table 2
Various scales of instance groups.

Instance groups	No. of orders ($ O $)	No. of SKUs ($ I $)	No. of picking stations ($ S $)	No. of replenishment stations ($ \bar{S} $)	No. of pods ($ K $)	No. of robots ($ R $)
ISG1	5	2	3	1	2	4
ISG2	8	2	4	1	2	4
ISG3	10	3	4	1	3	5
ISG4	15	5	4	1	3	5
ISG5	15	6	5	1	4	6
ISG6	15	8	5	1	6	8
ISG7	20	10	5	2	10	12
ISG8	25	10	6	2	12	14
ISG9	50	10	10	2	25	27

Table 3

Results of small-scale instances.

Case ID	CPLEX		VNS		Comparison GAP(%)
	F_{CPLEX}	$T_{\text{CPLEX}}(\text{s})$	F_{VNS}	$T_{\text{VNS}}(\text{s})$	
ISG1	220.00	27.67	220.00	16.50	0.00
	232.00	29.55	232.00	18.82	0.00
	301.00	25.85	301.00	18.34	0.00
	235.00	30.14	235.00	17.69	0.00
	209.00	12.66	209.00	27.87	0.00
ISG2	498.00	82.08	498.00	31.77	0.00
	288.00	188.96	288.00	30.55	0.00
	343.00	229.14	343.00	42.50	0.00
	521.00	426.94	521.00	29.21	0.00
	359.00	495.49	364.00	37.09	1.39
ISG3	197.00	1115.06	203.00	48.17	3.05
	331.00	1615.93	337.00	61.49	1.81
	302.00	>7200.00	319.00	43.30	5.63
	353.00	>7200.00	363.00	35.75	2.83
	374.00	>7200.00	379.00	44.77	1.34
Average	317.53	–	320.80	33.05	1.07

Note: (1) F_{CPLEX} (and F_{VNS}): the objective value of CPLEX (and VNS); (2) T_{CPLEX} (and T_{VNS}): the computation time of CPLEX (and VNS); (3) $\text{GAP} = (F_{\text{VNS}} - F_{\text{CPLEX}})/F_{\text{CPLEX}}$, represents the percentage difference between the objective values obtained by CPLEX and VNS.

Table 4

Results of medium-scale instances.

Case ID	Rule		VNS		Comparison GAP(%)
	F_{Rule}		F_{VNS}	$T_{\text{VNS}}(\text{s})$	
ISG4	685.00		493.00	92.23	38.95
	414.00		301.00	166.15	37.54
	490.00		396.00	85.58	23.74
	569.00		456.00	73.91	24.78
	436.00		418.00	53.26	4.31
ISG5	558.00		472.00	240.69	18.22
	728.00		592.00	184.61	22.97
	661.00		507.00	170.14	30.37
	675.00		588.00	219.46	14.80
	658.00		536.00	254.67	22.76
ISG6	596.00		458.00	362.16	30.13
	500.00		392.00	292.22	27.55
	516.00		427.00	356.57	20.84
	538.00		492.00	357.13	9.35
	505.00		420.00	264.75	20.24
Average	568.60		463.20	211.57	23.10

Note: (1) F_{Rule} (and F_{VNS}): the objective value of Rule (and VNS); (2) T_{VNS} , the computing time of VNS algorithm; (3) $\text{GAP} = (F_{\text{Rule}} - F_{\text{VNS}})/F_{\text{VNS}}$, represents the percentage difference between the objective values obtained by Rule and VNS.

the number of picking stations. The sensitivity analysis is conducted using three instances—ISG2, ISG4, and ISG7—which represent small, medium, and large-scale scenarios, respectively. The findings are illustrated in Fig. 2 and Fig. 3.

6.3.1. Sensitivity analysis on the order composition

We explore how the number of Stock Keeping Units (SKUs) in an order influences system performance. By varying the number of SKUs per order, we assess the system's responsiveness to changes in order complexity. For each problem instance group (ISG2, ISG4, and ISG7), we perform experiments with orders containing one to five different SKUs.

The data presented in Fig. 2 reveal a significant correlation between the maximum completion time and the number of SKUs per order. Specifically, as the number of SKUs increases, the maximum completion time exhibits an upward trend. For instance, in the large-scale instance ISG7, the maximum completion time increases from 253 units to 945 units when the number of SKUs per order rises from 1 to 5. This trend is attributed to the increased demand for diverse SKUs, which necessitates the involvement of more pods to fulfill each order. Consequently, the system experiences longer picking operations and increased travel times for robots and pods.

This finding highlights the sensitivity of the RMFS to order complexity. Orders with a higher number of SKUs require more intricate coordination among robots, pods, and stations, leading to potential delays. From a managerial perspective, controlling the number of SKUs per order or grouping orders with similar SKUs can enhance system efficiency and reduce completion times.

Table 5
Results of large-scale instances.

Case ID	Rule F_{Rule}	VNS F_{VNS}	$T_{VNS}(s)$	Comparison GAP(%)
ISG7	888.00	707.00	432.22	25.60
	848.00	622.00	580.35	36.33
	680.00	536.00	501.33	26.87
	849.00	633.00	456.61	34.12
	683.00	577.00	550.25	18.37
ISG8	812.00	616.00	667.18	31.82
	768.00	543.00	759.58	41.44
	809.00	643.00	609.72	25.82
	839.00	711.00	756.58	18.00
	759.00	671.00	683.46	13.11
ISG9	1073.00	844.00	1218.79	27.13
	1560.00	1229.00	1655.98	26.93
	1477.00	1077.00	1274.22	37.14
	1087.00	933.00	1760.42	16.51
	1240.00	901.00	1018.67	37.62
Average	958.13	749.53	861.69	27.79

Note: (1) F_{Rule} (and F_{VNS}): the objective value of Rule (and VNS); (2) T_{VNS} : the computing time of VNS algorithm; (3) $GAP = (F_{Rule} - F_{VNS})/F_{VNS}$, represents the percentage difference between the objective values obtained by Rule and VNS.

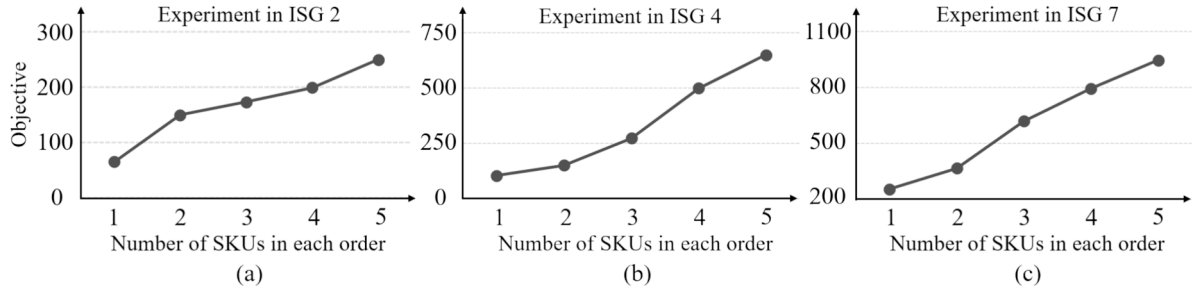


Fig. 2. Sensitivity analysis of the order composition.

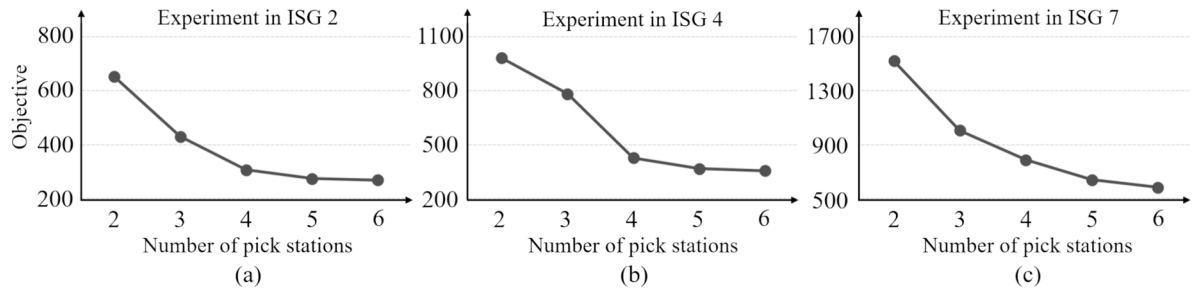


Fig. 3. Sensitivity analysis of the number of picking stations.

6.3.2. Sensitivity analysis on the number of picking stations

The number of picking stations is a critical factor that can significantly affect system performance. To investigate this, we analyze how varying the number of picking stations from 2 to 6 impacts the maximum completion time in the three instance groups, while keeping other parameters constant.

As illustrated in Fig. 3, there is a clear inverse relationship between the number of picking stations and the maximum completion time. Increasing the number of picking stations from 2 to 4 results in a notable reduction in completion time—by approximately 50 % on average across all instance groups. However, beyond four stations, the marginal benefit diminishes. For example, adding a fifth or sixth station yields only a marginal decrease in completion time, suggesting diminishing returns.

In the medium-scale instance ISG4, increasing the picking stations from 2 to 4 reduces the maximum completion time from 982 units to 428 units. However, further increasing the number to 6 stations only reduces the completion time to 356 units. This indicates that an optimal number of picking stations exists beyond which additional stations do not contribute significantly to performance.

improvement. Moreover, having too many picking stations may lead to underutilization of resources, as each station handles fewer orders.

These results suggest that managers should carefully balance the number of picking stations to optimize resource utilization and system efficiency. Investing in additional stations beyond the optimal point may not yield proportional benefits and could lead to unnecessary operational costs.

6.4. Experimental summary

This section presents comprehensive numerical experiments to assess the effectiveness of the Variable Neighborhood Search (VNS) algorithm in solving the integrated scheduling problems in RMFS.

We conducted a series of experiments involving small, medium, and large-scale instances. The results demonstrate that the VNS algorithm is highly effective across different scales. For small-scale instances, the VNS algorithm produces solutions comparable to those obtained by CPLEX but with significantly faster computation times. On medium-scale instances, the VNS algorithm consistently outperforms heuristic rules in terms of solution quality. For large-scale instances, the VNS algorithm offers stable and effective solutions, maintaining solution quality while keeping computation times within practical limits.

The sensitivity analysis provides valuable insights into how key parameters affect system performance. Specifically, the analysis reveals that the maximum completion time is highly sensitive to the number of SKUs per order and the number of picking stations. An increase in SKUs per order leads to longer completion times due to increased operational complexity and resource requirements. Conversely, increasing the number of picking stations reduces completion times up to a point, after which additional stations offer minimal benefits.

These findings have important managerial implications. Warehouse managers can optimize the efficiency of RMFS by balancing order composition and resource allocation. For instance, grouping orders to minimize the number of SKUs per order or strategically limiting the diversity of SKUs can improve system performance. Additionally, determining the optimal number of picking stations is crucial to ensure resource utilization without incurring unnecessary operational costs.

In summary, the experiments validate the effectiveness of the VNS algorithm and highlight critical factors that influence RMFS performance. The insights gained from the sensitivity analysis can guide warehouse managers in making informed decisions to enhance operational efficiency, demonstrating the practical applicability of our integrated optimization approach.

7. Conclusion

This paper aims to enhance the service quality of the RMFS by optimizing the allocation of order-to-station, order-to-pod, and robot-to-pod and plans the travel route of robots and pods. To achieve the objective of minimizing the longest completion time, a five-stage MILP model is proposed. Additionally, a VNS algorithm is developed to efficiently solve the model for instances of different scales. The algorithm's effectiveness is confirmed through numerical experiments, while sensitivity analysis offers practical insights for managers. The following contributions were achieved in this study:

(1) From the perspective of modelling, this paper proposes a more comprehensive five-stage MILP model. It not only covers the assignment of orders, the matching of robots and pods, and the replenishment and picking routes but also deeply considers the sequence of each piece of equipment and operating processes. Only a limited number of studies have been found to consider all of these factors simultaneously.

(2) From the perspective of the algorithm, we present a highly efficient VNS algorithm and validate its practicality through numerous numerical experiments. Experimental data indicates that in small-scale instances, the VNS algorithm is more effective in finding solutions than the CPLEX solver. The VNS algorithm not only outperforms Rule in medium and large-scale examples but also consistently maintains a stable level of efficiency across a wide range of computational instances.

(3) From the perspective of application, we have gained managerial insights from sensitivity experiments. For instance, the completion time is significantly affected by the number of SKUs in order. As the number of SKUs in an order increases, the replenishment and picking tasks also increase correspondingly, further extending the required operational time. The number of picking stations also has a big impact on the system's performance. Therefore, it is crucial to carefully select the appropriate number of picking stations, considering the system's efficiency, cost, and specific operational requirements. An excessive number of picking stations will have minimal impact and lead to unnecessary waste.

Although this study contributes to the optimization of the RMFS, there remains a need for further investigation of certain factors. Regarding mathematical modelling, we plan to address more complex issues such as the integrated replenishment problem combined with RMFS and the charging problem. Additionally, we aim to design exact algorithms that achieve improved computation speeds across various RMFS models. Warehouse logistics are expected to become increasingly mobile, modular, and flexible in the future. Consideration must be given to logistics and transportation issues in dynamic scenarios, incorporating unforeseen events such as robot failures and temporary order insertion. Our future research will delve into problems associated with multiple scenarios, such as temporary order insertions and robot malfunctions, ultimately providing advanced decision support.

CRedit authorship contribution statement

Jingwen Wu: Writing – original draft, Software, Methodology, Data curation. **Zhiyuan Yang:** Writing – original draft, Software, Methodology, Investigation, Data curation. **Lu Zhen:** Writing – review & editing, Supervision, Methodology, Investigation, Funding

acquisition, Conceptualization. **Wenxin Li:** Writing – original draft, Software, Methodology, Investigation. **Yiran Ren:** Writing – review & editing, Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research is supported by the National Natural Science Foundation of China (Grant numbers 72025103, 72394360, 72394362, 72361137001).

Appendix A. Linearization of the model

In the model, Constraints (8), (49), (56) and (58) are nonlinear as they contain the product of two variables. To linearize these constraints, some additional notations are defined below. Constraint (8) is replaced by Constraints (63)–(67), Constraint (49) by Constraints (68)–(72), and Constraints (56) and (58) by Constraints (73)–(82).

Index and set

D set of feasible values denoting a number of SKU units of a pod picked at a station, with an index of d , $d \in \{1, \dots, |D|\}$.

Decision variables

ϖ_{osk} binary, if the pod k is picked for order o at the station s is equal to one, and otherwise zero.

ψ_{oski} binary, if the SKU i in the pod k is picked for the order o at the station s is equal to one, and otherwise zero.

q_{oski} number of the unit of SKU i of the pod k picked at the station s for the order o ;

ζ_{dski} binary, if a number of SKU i 's units of the pod k picked at the station s is d , the value is equal to one, and otherwise zero.

Constraints

$$\varpi_{osk} \leq \alpha_{os} \quad \forall o \in O, s \in S, k \in K \quad (63)$$

$$\varpi_{osk} \leq \beta_{ok} \quad \forall o \in O, s \in S, k \in K \quad (64)$$

$$\varpi_{osk} \geq \alpha_{os} + \beta_{ok} - 1 \quad \forall o \in O, s \in S, k \in K \quad (65)$$

$$\min \left\{ \sum_{o \in O} \varpi_{osk}, 1 \right\} = \delta_{sk} \quad \forall s \in S, k \in K \quad (66)$$

$$\varpi_{osk} \in \{0, 1\} \quad \forall o \in O, s \in S, k \in K \quad (67)$$

$$\psi_{oski} \leq \alpha_{os} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (68)$$

$$\psi_{oski} \leq \xi_{oki} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (69)$$

$$\psi_{oski} \geq \alpha_{os} + \xi_{oki} - 1 \quad \forall o \in O, s \in S, k \in K, i \in I \quad (70)$$

$$\rho_{ks} = \sum_{i \in I} \min \left\{ \sum_{o \in O} \psi_{oski}, 1 \right\} \quad \forall k \in K, s \in S \quad (71)$$

$$\psi_{oski} \in \{0, 1\} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (72)$$

$$q_{oski} \leq \gamma_{oki} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (73)$$

$$q_{oski} \leq M\alpha_{os} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (74)$$

$$q_{oski} \geq \gamma_{oki} - M(1 - \alpha_{os}) \quad \forall o \in O, s \in S, k \in K, i \in I \quad (75)$$

$$q_{oski} \in \{0, 1\} \quad \forall o \in O, s \in S, k \in K, i \in I \quad (76)$$

$$\sum_{o \in O} q_{oski} \geq \sum_{d \in D} (2d - 1) \zeta_{dski} \quad \forall s \in S, k \in K, i \in I \quad (77)$$

$$\sum_{o \in O} q_{oski} \leq \sum_{d \in D} 2d \zeta_{dski} \quad \forall s \in S, k \in K, i \in I \quad (78)$$

$$\sum_{d \in D} \zeta_{dski} = 1 \quad \forall s \in S, k \in K, i \in I \quad (79)$$

$$\zeta_{dski} \in Z^+ \quad \forall d \in D, s \in S, k \in K, i \in I \quad (80)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F\left(i, \sum_{o \in O} \varrho_{oski}\right) + t_{ss}^2 \leq \mu_{ks'} + M(1 - \theta_{kss'}) \quad (81)$$

$$\nu_{ks} + a\rho_{ks} + \sum_{i \in I} F\left(i, \sum_{o \in O} \varrho_{oski}\right) \leq \nu_{ks'} + M(1 - \varphi_{skk'}) \quad (82)$$

Appendix B. VND framework

Algorithm 2: VND framework

Input: Initial solution π' , best found solution π , total number of neighborhoods l_{max} 1: **While** $l \leq l_{max}$
do2: **Switch** l **do**3: **Case 1** Performing the *sequencing exchange strategy* to obtain the new order allocation scheme on the given initial solution π' , and solving Stage 2–5 to obtain the new scheme π'' and the objective value $f(\pi'')$ 4: **break**;5: **Case 2** Performing the *pairwise exchange strategy* to obtain the new order allocation scheme on the given initial solution π' , and solving Stage 2–5 to obtain the new scheme π'' and the objective value $f(\pi'')$ 6: **break**;7: **Case 3** Performing the *mirror inversion strategy* to obtain the new pod's travel routes on the given initial solution π' , and solving Stage 5 to obtain the new scheme π'' and the objective value $f(\pi'')$ 8: **break**;9: **Case 4** Performing the *parallel movement strategy* to obtain the new pod's travel routes on the given initial solution π' , and solving Stage 5 to obtain the new scheme π'' and the objective value $f(\pi'')$ 10: **break**;11: **Default**12: **break**;13: **End switch**14: **If** $(f(\pi'') < f(\pi))$
then15: $\pi = \pi'', \pi' = \pi'', l = 0$;16: **Else**17: $l = l + 1$;18: **End if**19: **End while**20:
Return $\pi, f(\pi)$

The VND framework employs four neighborhoods in a local search to improve solution quality. Algorithm 2 presents the pseudo-code. It starts by exploring a neighborhood using the first strategy to generate a new solution. If the new solution is not better, it moves to the next neighborhood. When a better solution is found, it returns to the first neighborhood and repeats the process. If no better solution is found in all neighborhoods, the program will stop and the final solution will be output.

In addition, the algorithm uses CPLEX to compute the subproblems to compose the final solutions. In fact, the best solution in each subproblem is not necessarily the global best solution, so in the third and fourth stages of calculation for CPLEX, we set a certain tolerance gap between the best integer objective and the remaining best node objective to make the solution have a certain randomness. When the value falls below this parameter, mixed integer optimization will be terminated.

References

- Boysen, N., Briskorn, D., Emde, S., 2017. Parts-to-picker based order processing in a rack-moving mobile robots environment. *Eur. J. Oper. Res.* 262 (216), 550–562.
- Boysen, N., Schwerdfeger, S., Ulmer, M.W., 2023. Robotized sorting systems: Large-scale scheduling under real-time conditions with limited lookahead. *Eur. J. Oper. Res.* 310 (2), 582–596.
- Cergibozan, Ç., Tasan, A.S., 2019. Order batching operations: an overview of classification, solution techniques, and future research. *J. Intell. Manuf.* 30, 335–349.
- D'Andrea, R., 2012. Guest editorial: A revolution in the warehouse: A retrospective on kiva systems and the grand challenges ahead. *IEEE Trans. Autom. Sci. Eng.* 9 (4), 638–639.
- Dou, J., Chen, C., Yang, P., 2015. Genetic scheduling and reinforcement learning in multi-robot systems for intelligent warehouses. *Math. Probl. Eng.* 25, 1–10.
- Huang, M., Hao, Y., Wang, Y., Hu, X., Li, L., 2023. Split-order consolidation optimization for online supermarkets: Process analysis and optimization models. *Frontiers of Engineering Management* 10 (3), 499–516.
- Jiang, M., Leung, K.H., Lyu, Z., Huang, G.Q., 2020. Picking-replenishment synchronization for robotic forward-reserve warehouses. *Transp. Res. Part E: Logistics Transp. Res.* 144, 102138.
- Lamballais, T., Roy, D., De Koster, M.B.M., 2017. Estimating performance in a robotic mobile fulfillment system. *Eur. J. Oper. Res.* 256 (3), 976–990.
- Lamballais, T., Roy, D., De Koster, R.B., 2020. Inventory allocation in robotic mobile fulfillment systems. *IIE Trans.* 52 (1), 1–17.
- Masae, M., Glock, C.H., Grosse, E.H., 2020. Order picker routing in warehouses: A systematic literature review. *Int. J. Prod. Econ.* 224, 107564.
- Menéndez, B., Bustillo, M., Pardo, E.G., Duarte, A., 2017a. General variable neighborhood search for the order batching and sequencing problem. *Eur. J. Oper. Res.* 263 (1), 82–93.
- Menéndez, B., Pardo, E.G., Alonso-Ayuso, A., Molina, E., Duarte, A., 2017b. Variable neighborhood search strategies for the order batching problem. *Comput. Oper. Res.* 78, 500–512.
- Nicolas, L., Yannick, F., Ramzi, H., 2018. Order batching in an automated warehouse with several vertical lift modules: Optimization and experiments with real data. *Eur. J. Oper. Res.* 267 (3), 958–976.

- Roy, D., Nigam, S., de Koster, R., Adan, I., Resing, J., 2019. Robot-storage zone assignment strategies in mobile fulfillment systems. *Transp. Res. Part E Logistics and Transp. Rev.* 122, 119–142.
- Valle, C.A., Beasley, J.E., 2021. Order allocation, rack allocation and rack sequencing for pickers in a mobile rack environment. *Comput. Oper. Res.* 125, 105090.
- Van Gils, T., Ramaekers, K., Braekers, K., Depaire, B., Caris, A., 2018. Increasing order picking efficiency by integrating storage, batching, zone picking, and routing policy decisions. *Int. J. Prod. Econ.* 197, 243–261.
- Wang, Z., Sheu, J.B., Teo, C.P., Xue, G., 2022. Robot scheduling for mobile-rack warehouses: Human-robot coordinated order picking systems. *Prod. Oper. Manage.* 31 (1), 98–116.
- Wang, B., Yang, X., Qi, M., 2023. Order and rack sequencing in a robotic mobile fulfillment system with multiple picking stations. *Flex. Serv. Manuf. J.* 35, 509–547.
- Xie, L., Thieme, N., Krenzler, R., Li, H., 2021. Introducing split orders and optimizing operational policies in robotic mobile fulfillment systems. *Eur. J. Oper. Res.* 288 (1), 80–97.
- Xue, J., Li, G., 2023. Balancing resilience and efficiency in supply chains: Roles of disruptive technologies under Industry 4.0. *Front. Eng. Manage.* 10 (1), 171–176.
- Yang, X., Hua, G., Hu, L., Cheng, T.C.E., Huang, A., 2021. Joint optimization of order sequencing and rack scheduling in the robotic mobile fulfillment system. *Comput. Oper. Res.* 135, 105467.
- Zou, B., Gong, Y., Xu, X., Yuan, Z., 2017. Assignment rules in robotic mobile fulfillment systems for online retailers. *Int. J. Prod. Res.* 55 (20), 6175–6192.